

西南财经大学本科期末考试试题册（B）

（2024—2025学年第1学期）

以下各项由命题教师填写：

课程名称：数据结构（英） 命题教师：陈中普 王海林

适用对象（年级专业）：2023级信息管理与信息系统、2023级物流管理、2023级金融数学

使用试题的任课教师姓名：陈中普 王海林

试题说明：

- 1、考试类型：闭卷☒ 开卷☐
- 2、本套试题共 三 道大题，共 6 页，完卷时间 120 分钟。
- 3、考试用品中除纸、笔、尺子外，可另带的用具有：
计算器☐ 字典☐ 等
(请在下划线上填上具体数字或内容，所选☐内打钩)

考试时间（由制卷方填写）：

以下各项由学生填写：

任课教师：

年级专业：

学生姓名：

学 号：

- 考生注意事项：
- 1.出示学生证（或身份证）和准考证于桌面左上角，以备查验。
 - 2.拿到试卷后清点并检查试卷页数，如有重页、页数不足、空白页及印刷模糊等举手向监考教师示意调换试卷。
 - 3.答题前先将试题册及答题纸上的任课教师、专业、年级、学号、姓名填写完整。
 - 4.所有答案均需填写在答题纸上，答在试题册上无效。
 - 5.考生不得携带任何通讯工具进入考场。
 - 6.考试结束后，将试题册、答题纸和草稿纸等下发材料上交监考教师，不得带离考场。
 - 7.严格遵守考场纪律。

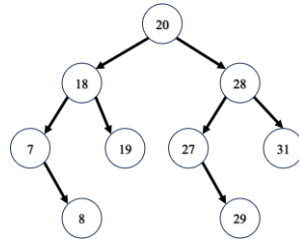
- We assume that the height of an empty tree is -1.
- The node in a singly linked list has attribute *data* and *next*, in which *data* is the element it is holding, and *next* is the link to the next node. If it is doubly linked list, the node holds *prev* link to the previous nodes.
- The node in a binary tree has attribute *key*, *left*, and *right*, in which *key* is the data it is holding, and *left* and *right* are links to its left and right child, respectively.

Part One: Multi-choices (2 marks each; 20 marks in total)

(Note: Each question has only one correct choice.)

1. What is the print result of the following series of operations on a stack. Push(1), Push(2), Peek(), Push(3), Pop(), Push(4), Push(5), if Peek() == 5 then Pop(), Push(6), Pop(), Pop(), Push(7), if Stack size == 3 then Pop(), Peek():
 A. 2, 3, 6, 5, 7
 B. 2, 3, 5, 6, 7
 C. 3, 2, 6, 5, 7
 D. 2, 3, 6, 7, 5
2. What values are returned during the following series of queue operations: enqueue(10), enqueue(20), dequeue(), enqueue(15), enqueue(9), dequeue(), enqueue(7), if queue size > 2 then dequeue(), enqueue(6), dequeue(), dequeue(). What is the sequence of returned values?
 A. 10, 20, 15, 7, 9,
 B. 10, 20, 9, 15, 6,
 C. 10, 20, 15, 9, 7
 D. 10, 15, 9, 7, 6
3. What is the time complexity of a LinkedList to remove the *k*-th to last element, if the LinkedList without the tail reference?
 A. $O(1)$
 B. $O(N)$
 C. $O(N^2)$
 D. $O(\log N)$
4. Which of the following statements is least characteristic of an ideal hash function, given the properties of a hash function in the context of a hash table?
 A. It should produce the same hash value for the same input (Deterministic).
 B. It should compute hash values quickly to maintain efficient insertion and search operations (Fast computation).
 C. It should distribute keys uniformly across the hash table to minimize collisions (Uniform distribution).
 D. It should be able to regenerate the original key from its hash value without any information loss (Reversible)

5. Given a BST as follow, if we first insert the value 5 as the left child of 7, and then delete the node with the value 7, how many comparisons are required to search for the value 8? (Note: during deletion, if a deleted node has two children, the smallest node in right sub-tree will be promoted to replace the deleted node.)



- A. 1
B. 2
C. 3
D. 4
6. In a complete binary tree of height 4, where each node contains a unique integer value from 1 to the maximum number of nodes in the tree, what is the sum of all node values if the tree is also a binary search tree (BST)?()
- A. 136
B. 128
C. 120
D. 139
7. Which of the following is not a valid method to resolve collisions in a hash table?
- A. Chaining
B. Resizing
C. Rehashing
D. Overwriting
8. Given the BST in Question 5, what is the post-order tree walk after inserting 5, deleting 8, inserting 35, inserting 9?
- A. 9 -> 5 -> 20 -> 19 -> 18 -> 29 -> 27 -> 35 -> 31 -> 28 -> 7
B. 5 -> 9 -> 7 -> 19 -> 29 -> 18 -> 27 -> 35 -> 31 -> 28 -> 20
C. 5 -> 9 -> 7 -> 18 -> 19 -> 27 -> 29 -> 31 -> 35 -> 28 -> 20
D. 5 -> 9 -> 7 -> 19 -> 18 -> 29 -> 27 -> 35 -> 31 -> 28 -> 20
9. Given a max-heap stored in an array [30, 28, 27, 16, 19, 21, 26, 3, 11], perform the following operations: (1) Remove the root element 30. (2) Insert the value 35 into the heap. (3) Insert the element 17 from the heap. After performing these operations, which of the following arrays correctly represents the max-heap?
- A. [35, 28, 27, 16, 19, 17, 26, 3, 11, 27]
B. [35, 28, 27, 19, 17, 21, 26, 3, 16, 11]
C. [28, 27, 35, 16, 11, 19, 26, 3, 11, 17]

D. [28, 35, 27, 16, 19, 26, 11, 3, 17, 21]

10. Suppose Bob hashes an array [18, 16, 17, 29, 12, 19, 20] using the separate chaining method with the hash function $h(\text{key}) = \text{key} \% 11$, and each chain is represented as a linked list. Before checking whether the hash table contains the number 7, Bob performs the following additional operations: (1) Inserts the number 7 into the hash table, (2) Removes the element with the smallest value in each chain. After these operations, how many times does Bob need to compare values to check whether the hash table contains the number 7?
- A. 1 comparison
 - B. 2 comparisons
 - C. 3 comparisons
 - D. 4 comparisons

Part Two: True or False (2 mark each; 20 marks in total)

1. A singly linked list with only a tail pointer cannot be reversed from the tail in $O(n)$ time. (True/False)
2. A doubly linked list can be searched in $O(n)$ time. Thus, any node can also be deleted in $O(n)$ time? (True/False)
3. If $f(n) = 5n^{20} + 28n^2 + 34n^8 + 9n^{200} + 30n^{25} + 5$, then $f(n) = O(n^2)$. (True/False)
4. Deque data structure via a singly linked list allows efficient insertion and deletion of elements at start or end positions. (True/False)
5. A BST using an in-order traversal will yield the elements in sorted order. (True/False)
6. Implementing a doubly linked list with special deletion, where deleted nodes are marked as unused, does not affect the ability to traverse the list efficiently. (True/False)
7. The time to remove the highest priority element from a binary heap remains $O(\log n)$ with heapifying. (True/False)
8. A sorted singly linked list is sorted in descending order and the time complexity of finding the k -th smallest element remains $O(n)$. (True/False)
9. Frequent resizing of a hash table due to a poor strategy will affect the search time complexity. (True/False)
10. In an undirected graph with cycles, finding a path from a single source node to a single target node using depth-first search require visiting all nodes. (True/False)

Part Three: Short Q&A (6 marks each; 24 marks in total)

1. If $f(n) = n!$, then $f(n) = O(n^n)$. Does it make sense? Explain.
2. In a graph data structure, an adjacency list is used to represent connections between nodes. What are the space and time trade-offs of using an adjacency list compared to an adjacency matrix?

3. The following code is supposed to implement a queue using nodes of a linked list. However, there is an error in the code that prevents it from working correctly. Please identify the bug and provide the corrected code.

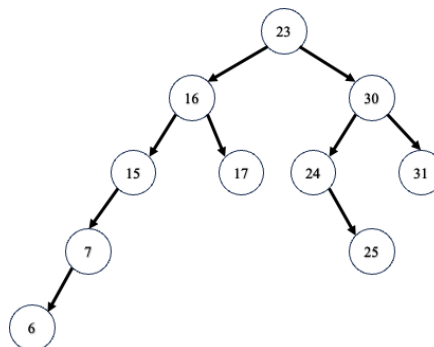
```
class Node:
```

```
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
class Queue:
```

```
    def __init__(self):
        self.front = None
        self.rear = None
    def enqueue(self, data):
        new_node = Node(data)
        self.rear = new_node
        if self.front is None:
            self.front = new_node
        else:
            self.rear.next = new_node
```

4. An AVL tree is a self-balancing binary search tree. However, in the following figure, the tree is unbalanced. Please identify the node that needs to be rotated and indicate whether it should be left-rotated, right-rotated, or both to restore the balance of the tree. Draw the modified tree after the rotation.



Part Four: Design and Analysis (36 marks in total)

(Note that pseudo-code and even plain English are allowed.)

1. Given a singly linked list with only the head, it is called "large" if at least half of its elements is greater than or equal to 10. For example, 3->10->1->4 is not large, while 10->10->8->12 is large. Please design *is_large()* to check whether it is large. (10 marks)
(Hint: it should return either True or False)

```
class LinkedList:
```

```
    def __init__(self):
        self._head = None
        self._rear = None
```

```
def is_large(self):
    pass # implement this
```

2. Given a binary search tree with a root node, please design an algorithm find_min() to find the minimum value in the tree. (10 marks) (Hint: it should return the minimum value or None if the tree is empty, and can't use sort methods.)

```
class BST:
    def __init__(self):
        self._root = None

    def find_min(self):
        pass # implement this
```

3. Given a singly sorted linked list only with the head node. Please design an algorithm remove_duplicates() to remove all duplicate elements while keeping it sorted. For example, after removing duplicates, 1->4->7->7->8->20 becomes 1->4->7->8->20. (10 marks)

```
class SinglyLinkedList:
    def __init__(self):
        self._head = None

    def remove_duplicates(self):
        pass # implement this
```

4. You can choose either A or B. (6 marks)

A. Given a graph represented as an adjacency list, design a function to check if a graph is connected. A graph is connected if there is a path between every pair of vertices. (Hint: you can use Depth-First Search or Breadth-First Search to check connectivity.)

```
class Graph:
    def __init__(self, vertices):
        self.vertices = vertices
        self.adjacency_list = {vertex: [] for vertex in range(vertices)}

    def add_edge(self, u, v):
        self.adjacency_list[u].append(v)
        self.adjacency_list[v].append(u)

    def is_connected(self):
        pass # implement this

    def _dfs(self, vertex, visted):
        pass # implement this
```

B. Consider the scenario where you are implementing an algorithm to find the middle element of a singly linked list.

- a. What is the time complexity of this algorithm in terms of the number of nodes in the list and provide an analysis?
- b. If the linked list is known to have an even number of nodes, how would you determine the two middle elements?
- c. How can you optimize the process if you have additional information about the list, such as its length?